PHP2Shell 2026:

Root Cause, Disclosure, Human Factors, Honeypot Evidence, Business Risk and PHP Ecosystem Resilience

Md Riyadh Hasan

Independent Researcher, Oxford, United Kingdom

1 November 2026

Rapid Technical Incident Report - Version 1.0.0

Copyright © 2026 Md Riyadh Hasan. All rights reserved.

This report is an independent analysis. It does not claim discovery, ownership, or authorship of PHP2Shell, CVE-2026-109618, CVE-2026-109619, PHP, or third-party research.

Executive Summary

Hypothesis focus: this report formally evaluates five hypotheses (H1-H5): structural failure, premature disclosure, human/social-engineering contribution, insider involvement, and honeypot-based timeline reconstruction. The insider hypothesis is not presented as a finding; it is currently unsupported by the public evidence reviewed.

PHP2Shell is the name used for two critical vulnerabilities in the PHP runtime, CVE-2026-109618 and CVE-2026-109619, publicly disclosed and patched on 21 October 2026. Both can permit unauthenticated remote code execution under vulnerable web-serving configurations, and both arise from unsafe runtime state around fatal-error and out-of-memory handling rather than from a particular content-management system or application plugin. PHP's own advisory identifies PHP-FPM, Apache mod_php, php-cgi, FrankenPHP and LiteSpeed as in scope, with mod_php described as the highest-risk configuration. The PHP project states that there are no complete mitigations and that updating to a fixed release is the safe response.

The incident is significant not only because of technical severity but because it exposes a concentration risk in the web ecosystem. PHP sits beneath large numbers of websites, content-management systems, e-commerce services, hosting environments and commercial applications. A defect in the runtime can therefore cross application boundaries: updating WordPress, Drupal, Joomla, Laravel or another PHP application does not by itself repair a vulnerable PHP interpreter. Businesses must identify and patch the runtime actually serving requests, restart long-lived workers, verify the deployed build, review logs, and assess whether credentials or secrets require rotation.

The available evidence does not support reducing PHP2Shell to a single person's mistake. The PHP Foundation's post-incident account says signs of the underlying out-of-memory problem had been identified during 2024 security work, but remediation stalled amid compatibility concerns, denial-of-service trade-offs, third-party extension considerations, consensus-based engineering and limited specialist capacity. The Foundation explicitly characterized the earlier handling as a structural failure and has since expanded professional security capacity. This report therefore treats PHP2Shell as a combined software-engineering, vulnerability-management and ecosystem-governance event.

The report also proposes the Hasan PHP Dependency Resilience Framework (PDRF), a practical five-layer model for organizations that depend on PHP. The framework prioritizes runtime inventory and patchability, exposure reduction, compromise containment, recoverability, and architectural optionality. Its central recommendation is not to abandon PHP reflexively, but to eliminate unsupported runtimes, reduce direct internet exposure, make recovery and secret rotation routine, and create an exit or diversification path for business-critical services whose risk cannot be accepted.

Key Findings

- PHP2Shell is a PHP runtime vulnerability family, not a WordPress-, Drupal-, Joomla-, or framework-specific flaw.
- PHP released fixed versions 8.5.12, 8.4.27, 8.3.36 and 8.2.35 on 21 October 2026 and advised immediate updating.

- Application authentication can be irrelevant to some demonstrated attack paths because vulnerable runtime processing may occur before normal application logic.
- Public reporting indicates exploitation attempts around the disclosure window; individual telemetry claims must be interpreted carefully because some early reports were corrected or retracted.
- The business risk extends beyond first-party systems to shared hosting, managed services, suppliers and third-party products that bundle PHP.
- PHP's security organization expanded materially during 2026, including an Ecosystem Security Team and additional security-focused contributors.
- Leadership changes at the PHP Foundation occurred in 2026, but there is no evidence that those personnel changes caused PHP2Shell.
- Long-term resilience requires both patch discipline and architectural recovery options; replacing PHP everywhere is neither necessary nor a complete security strategy.

1. Incident Timeline

Date	Event
2024	During security work/audit activity, the PHP ecosystem identified signs of the underlying out-of-memory problem. A practical exploitation route had not yet been established, and remediation did not reach completion.
30 Jul 2026	Security firm Calif began an audit of PHP under the Patch the Planet program, according to the PHP Foundation.
11 Aug 2026	Calif researcher Ryan A. reported unconditional RCE with the same root cause. The PHP Foundation also says OpenAI independently reported the root cause.
19-24 Aug 2026	Wiz says it discovered the first vulnerability on 19 August, confirmed unauthenticated RCE against Drupal on 20 August and WordPress on 21 August, and reported the vulnerabilities to PHP on 24 August.
Sep 2026	Additional attack vectors and hardening work continued while PHP coordinated a fix. Daniel Scherzer joined the PHP Foundation's Ecosystem Security Team in September.
20 Oct 2026	Information about the CVEs became public ahead of the planned coordinated disclosure. The PHP Foundation moved its

	communications/release response forward.
21 Oct 2026	PHP released 8.5.12, 8.4.27, 8.3.36 and 8.2.35, fixing CVE-2026-109618 and CVE-2026-109619.
22-29 Oct 2026	Security vendors and researchers reported widespread scanning and exploitation attempts. Unit 42 later corrected an earlier attribution and, on 29 October, reported a campaign chaining PHP2Shell with a Linux kernel privilege-escalation vulnerability against internet-exposed PAN-OS management interfaces.
1 Nov 2026	This report consolidates the technical, organizational and business-risk evidence available to date.

2. What PHP2Shell Is

PHP2Shell is a label for two related critical memory-safety/state-management vulnerabilities in PHP core. CVE-2026-109618 concerns corruption associated with out-of-memory handling: fatal interruption can occur while internal request data is only partially initialized, after which cleanup logic may treat invalid state as valid. CVE-2026-109619 concerns stale request-owned global state that can survive after backing memory has been freed and then be used by a later request handled by the same worker. Both were rated critical in public advisories.

The important defensive point is location: the vulnerable behavior is inside the PHP engine. A secure application, current CMS plugin set, or strong login policy does not automatically remove runtime exposure. Researchers used WordPress, Drupal and Joomla as representative applications, but the PHP project and researchers emphasize that the vulnerable component is PHP itself.

This report intentionally does not reproduce weaponized exploit instructions. For defenders, the relevant mechanics are that abnormal memory pressure or fatal-error paths can leave request or global state in an unsafe condition; subsequent processing can transform that state corruption into memory disclosure or code execution under particular configurations and request sequences.

2A. Research Hypotheses and Evidence Status

Research question: Was PHP2Shell purely a technical runtime vulnerability, or did disclosure-process and human/organizational factors materially increase exploitation risk?

H1 - Structural-failure hypothesis. PHP2Shell was primarily the result of a difficult technical defect interacting with delayed remediation, compatibility constraints, vulnerability-handling limitations, coordination challenges, and finite specialist security resources. Current assessment: SUPPORTED and the strongest explanation in the evidence reviewed.

H2 - Disclosure-leak hypothesis. Premature circulation of vulnerability or CVE information may have increased attacker awareness and accelerated scanning or exploitation around coordinated disclosure. Current assessment: PLAUSIBLE / PARTLY SUPPORTED. Pre-disclosure observations support asking this question, but they do not establish who disclosed information or whether the same vulnerability was independently rediscovered.

H3 - Human/social-engineering hypothesis. Human factors - including accidental information exposure, compromised communications, social engineering, or access to non-public vulnerability information - may have contributed to premature attacker knowledge. Current assessment: OPEN / UNPROVEN. Pre-disclosure activity justifies investigation, but it does not identify the source of the information.

H4 - Insider-involvement hypothesis. A trusted participant could theoretically have intentionally or unintentionally disclosed non-public information that enabled early exploitation. Current assessment: UNSUPPORTED. No reliable public evidence reviewed for this report implicates a named PHP contributor, Foundation employee, researcher, recent joiner, departing member, or vendor. Personnel timelines are contextual evidence only and must not be used as proof of wrongdoing.

H5 - Honeypot-observation hypothesis. Distributed, isolated honeypots can help reconstruct whether PHP2Shell activity moved from generic scanning to exploit-like behavior before and after disclosure. The proposed study should use only researcher-owned or explicitly authorized systems, block harmful egress, contain no real customer data or reusable secrets, and publish defensive indicators rather than weaponized exploit instructions.

ID	Hypothesis	Current assessment	Evidence needed / falsification
H1	Structural/process failure	SUPPORTED	Primary chronology, remediation records and resource/process evidence.
H2	Premature disclosure accelerated hostile activity	PLAUSIBLE / PARTLY SUPPORTED	Independent pre-disclosure telemetry and evidence of the information path.
H3	Human/social-engineering contribution	OPEN / UNPROVEN	Communication compromise, phishing artifacts, access logs or reliable testimony.
H4	Intentional insider involvement	UNSUPPORTED	Reliable evidence tying a trusted participant to intentional disclosure or assistance.
H5	Honeypot timeline	SUPPORTED AS	Independent sensors

	reconstruction	METHOD	measuring probes, exploit-like traffic and post-exploitation behavior.

Evidence rule: personnel joining or leaving the PHP Foundation/security effort is contextual information only. Temporal proximity is not evidence of causation or wrongdoing. The report uses the categories Confirmed, Corroborated, Reported, Hypothesis and Unsupported to prevent speculation from being presented as fact.

3. Who Is at Risk?

PHP's official advisory identifies web-serving installations including PHP-FPM, Apache with mod_php, php-cgi, FrankenPHP and LiteSpeed. The project considers mod_php the highest-risk deployment model because an unauthenticated exploit was demonstrated before application code executes. The fixed upstream releases are PHP 8.5.12, 8.4.27, 8.3.36 and 8.2.35. PHP 8.1 and older are outside normal upstream support, although vendors may provide backports; PHP also provided minimal source backports for 7.4, 8.0 and 8.1 because of the incident's severity.

Organizations should not infer safety from a hidden PHP version header. Nor should they assume that upgrading WordPress, Drupal, Joomla, Magento, Laravel, Symfony or another application upgrades the PHP runtime. The runtime used by the web server may also differ from the command-line PHP version an administrator sees in a shell.

4. Human Error, Software Defect, or Structural Failure?

The strongest evidence supports a layered answer. At the code level, PHP2Shell is a software defect involving unsafe state during exceptional memory/error handling. At the engineering-process level, the PHP Foundation describes earlier handling of the OOM issue as a structural failure: a known class of problem did not reach a durable fix before later researchers demonstrated severe exploitation. At the ecosystem level, the security team faced limited specialist capacity, a growing volume of reports, compatibility concerns and the difficulty of changing behavior in a runtime used by a vast and diverse software ecosystem.

It would therefore be misleading to blame a named developer or executive without evidence. Open-source runtime security is a socio-technical system: code review, maintainership capacity, compatibility expectations, disclosure coordination, funding, testing and release engineering all affect whether a defect is found and fixed. PHP2Shell demonstrates how a technically subtle bug can become a systemic risk when the affected component is foundational.

5. PHP Security and Organizational Changes in 2026

The PHP Foundation had already begun expanding security capacity before public disclosure. In May 2026 it created an Ecosystem Security Team with Alpha-Omega/OpenSSF support and appointed Volker Dusch as Ecosystem AI Security Engineer in Residence. The team was intended to

help maintainers triage vulnerability reports, use modern security tooling, coordinate disclosure and support projects with limited maintainer capacity.

Alexandre Daubois joined the Foundation in July for security-report triage, handling and fixes. Daniel Scherzer joined the Ecosystem Security Team in September for PHP hardening work. The Foundation has also described using AI-assisted security tools alongside sanitizers and reproduction environments, while emphasizing that human reviewers make decisions and patches.

Leadership also changed during 2026. Elizabeth Barron became Executive Director in February, succeeding founding Executive Director Roman Pronskiy, who initially remained on the board. In September, Roman's board term ended and Brent Roose joined as the JetBrains representative. These are documented governance changes, but this report finds no evidence that they caused PHP2Shell. They should be understood as context for the Foundation's evolving capacity, not as attribution.

6. What Happened After Disclosure

The public-disclosure window moved quickly. PHP's own advisory urged immediate updates and stated that no complete mitigation substitutes for patching. Security vendors reported scanning and exploit attempts shortly after information became public, and some telemetry suggested activity even before coordinated disclosure. However, incident reporting evolved rapidly: at least one early claim of successful exploitation was later corrected, and Unit 42 explicitly retracted an earlier attribution before publishing a later, more specific campaign assessment. This is a reminder that 'attack traffic observed', 'exploit attempt', 'successful RCE' and 'confirmed production compromise' are different evidentiary categories.

For defenders, the practical conclusion is unchanged: treat vulnerable internet-facing PHP as an emergency patching condition. Where exploitation is suspected, patching alone is not incident response. Organizations should preserve relevant logs, inspect web/WAF and process activity, examine persistence and outbound connections, rotate exposed secrets when justified, and rebuild from trusted images where compromise cannot be excluded.

7. Internet-Business Risk and Economic Exposure

PHP2Shell matters economically because PHP is infrastructure beneath a large web ecosystem. The risk is not confined to technology companies. PHP-backed services can include e-commerce storefronts, customer portals, government and education sites, media platforms, healthcare portals, logistics systems, internal business applications, managed hosting and third-party SaaS products. A runtime-level pre-authentication RCE can therefore create business consequences before the application itself has a chance to enforce authentication or authorization.

Risk	Business effect
Service continuity	Emergency patching, restarts or precautionary shutdowns can interrupt customer-facing services and revenue-generating workflows.

Confidentiality	Successful memory disclosure or RCE may expose credentials, API keys, session material, configuration secrets or customer information present in a worker's process context.
Integrity	RCE can allow modification of application code, web content, configuration, scheduled tasks or data, depending on the web-server account's privileges.
Supply chain	A company may be patched internally yet remain exposed through agencies, managed hosting, SaaS suppliers, appliances or vendor products that embed PHP.
Compliance	Confirmed access to personal or regulated information can trigger contractual, legal, notification and forensic obligations depending on jurisdiction and data type.
Reputation	Defacement, prolonged downtime, customer notification or public disclosure of compromise can damage trust.
Legacy concentration	Unsupported PHP versions and applications that cannot tolerate runtime upgrades convert a patching problem into an architectural debt problem.

Vendor telemetry should be used cautiously when estimating global impact. For example, Check Point reported exploitation attempts affecting a notable share of monitored organizations, but explicitly defined 'impacted' as observing attempts rather than confirming successful compromise. Such data is useful for urgency, not as a count of breached organizations.

8. Who Depends on PHP - and What Dependency Really Means

PHP dependency exists at several layers. Direct dependency means an organization operates a PHP runtime itself. Platform dependency means it runs software such as WordPress, Drupal, Joomla, MediaWiki, Magento or PHP frameworks. Outsourced dependency means a hosting provider, digital agency or SaaS supplier operates PHP on the organization's behalf. Embedded dependency means PHP is present inside a commercial appliance or management interface that the customer may not think of as a 'PHP server'.

This distinction matters because ownership determines who can patch. A security team can upgrade an internally managed runtime immediately, but a shared-hosting customer may need the provider to act. A procurement team may need to obtain a vendor attestation for an appliance. A business that cannot identify which suppliers run PHP has a visibility problem before it has a patching problem.

9. Hasan PHP Dependency Resilience Framework (PDRF)

To convert the incident into a reusable business-security model, this report proposes the Hasan PHP Dependency Resilience Framework (PDRF). PDRF is an original analytical framework in this report; it is not a PHP Foundation standard. It organizes resilience into five layers that can be assessed independently and improved over time.

PDRF layer	Control objective
P1 - Runtime Visibility & Patchability	Maintain an authoritative inventory of PHP runtimes, versions, SAPIs, containers, base images, hosting providers and embedded/vendor instances. Define a maximum patch SLA for critical runtime CVEs and eliminate unsupported branches.
P2 - Exposure Control	Minimize direct origin exposure; use reverse proxies/WAFs as supplementary controls, restrict administrative interfaces, segment management planes, and remove unnecessary internet-facing PHP endpoints.
P3 - Compromise Containment	Run PHP with least privilege, separate secrets, restrict filesystem and network access, isolate tenants/workloads, monitor child processes and outbound connections, and prevent one web worker from becoming a path to the wider environment.
P4 - Recovery & Evidence	Maintain tested backups, immutable or offline recovery copies, infrastructure-as-code or reproducible images, centralized logs, secret-rotation procedures and a rehearsed rebuild process. Backups must be protected from the same credentials and hosts they are intended to recover.
P5 - Architectural Optionality	For business-critical services, document how the organization would upgrade, isolate, replace or migrate a PHP component if a future runtime defect cannot be patched quickly. Optionality may include service decomposition, API boundaries, container portability, or selective migration to another supported stack.

10. Backup and Recovery Strategy for PHP-Dependent Businesses

Backups are essential but they do not prevent exploitation. Their purpose is to make recovery possible after compromise, ransomware, destructive persistence or an emergency rebuild. A resilient PHP business should back up application data, configuration, deployment manifests and necessary content while avoiding blind restoration of potentially compromised executable code.

- Use a 3-2-1-style recovery principle: multiple copies, more than one storage medium or failure domain, and at least one protected/offline or immutable copy.
- Back up databases with transaction-consistent methods appropriate to the database engine and test restoration, not merely backup creation.
- Keep application source and infrastructure configuration in controlled version repositories so servers can be rebuilt rather than trusted after compromise.
- Separate backup credentials from production web-server credentials and prevent the PHP worker account from deleting or rewriting backup history.
- Maintain a documented secret-rotation inventory covering database credentials, API keys, OAuth secrets, certificates, session keys and deployment tokens.
- Record recovery-time and recovery-point objectives for customer-facing systems, and test them.
- After a suspected RCE, prefer rebuilding from a known-good patched image and restoring validated data over simply cleaning the existing host.

11. Should Businesses Replace PHP?

PHP2Shell is not evidence that every organization should migrate away from PHP. Every major runtime and framework can experience critical vulnerabilities, and rushed rewrites create new defects, operational risk and cost. The better decision is based on supportability, patch latency, architecture, business criticality and the organization's ability to contain a compromised web tier.

Option	When it may fit
Modern supported PHP	Best when the application is healthy, maintainable and patchable. Upgrade the runtime, automate patch testing and improve isolation.
Java / Spring	Suitable for large enterprise services where mature JVM operations and long-lived service architectures are already organizational strengths.
.NET	Strong option in Microsoft-centric enterprise environments with established identity, tooling and operations.
Node.js / TypeScript	Useful for JavaScript/TypeScript teams and event-driven web/API services, but still requires disciplined dependency and runtime patching.

Go	Attractive for small deployable services, networking components and operational simplicity; migration cost may be high for CMS-heavy workloads.
Python / Django / FastAPI	Strong ecosystem for APIs, automation and data-heavy services; not inherently safer merely because the language changes.
Rust	Useful where memory safety and systems-level control are major requirements, but adoption cost and developer availability can be constraints.

The strategic principle is diversification where it creates business value, not technology churn. An organization whose revenue depends on one large legacy PHP application should first make that application patchable and recoverable. It can then use service boundaries to move selected high-risk or high-change components to other stacks over time if justified.

12. Immediate Defensive Action Plan

1. Inventory every web-serving PHP runtime, including containers, staging, shared hosting and third-party products.

2. Upgrade to PHP 8.5.12, 8.4.27, 8.3.36 or 8.2.35, or install the security-fixed package supplied by the operating-system/LTS vendor.

3. Restart PHP-FPM pools, Apache/mod_php, FrankenPHP, LiteSpeed workers and other long-lived PHP-serving processes so old vulnerable code is no longer resident.

4. Verify the runtime actually serving HTTP requests rather than relying only on the CLI version.

5. Identify unsupported PHP branches and create an emergency migration or vendor-backport plan.

6. Review web/WAF and system telemetry from at least the pre-disclosure period onward where exposure existed; distinguish scanning from confirmed execution.

7. Where compromise indicators exist, preserve evidence, isolate affected systems, investigate persistence and outbound activity, rotate exposed secrets as appropriate, and rebuild from trusted patched images.

8. Contact hosting providers and critical suppliers for patch status where PHP is not under direct control.

9. Treat WAF rules and configuration restrictions as defense-in-depth, not substitutes for the PHP security update.

10. Adopt PDRF as a recurring quarterly control review for critical PHP-dependent services.

13. Lessons for Open-Source Security Governance

PHP2Shell illustrates a wider problem facing foundational open-source projects. AI-assisted security research can increase both useful vulnerability discovery and low-quality report volume. Maintainers therefore need triage capacity, reproducible testing, funded security engineering, clear disclosure processes and mechanisms to turn old warnings into tracked remediation work. The PHP Foundation's 2026 expansion of its security team is a direct response to that environment.

For businesses, the lesson is equally important: dependency on open-source software is not free of operational responsibility. Organizations that derive substantial commercial value from foundational runtimes should budget for patch engineering, supplier assurance and, where appropriate, financial support for the ecosystems on which they depend.

14. Conclusion

PHP2Shell should be understood as both a critical vulnerability event and a resilience test for the modern web. The technical defect was severe: vulnerable PHP runtimes could, under demonstrated conditions, permit unauthenticated memory disclosure or remote code execution. But the broader significance lies in the interaction between a foundational runtime, enormous downstream dependency, limited security capacity, disclosure pressure and legacy deployment.

The appropriate response is neither complacency nor an indiscriminate abandonment of PHP. Organizations should patch immediately, verify the serving runtime, investigate exposed systems, strengthen least privilege and segmentation, protect secrets, and maintain tested recovery. At the strategic level, the Hasan PHP Dependency Resilience Framework proposed here provides a way to reduce concentration risk through visibility, exposure control, containment, recoverability and architectural optionality.

Future research should measure real-world patch latency across hosting models, quantify third-party PHP dependency, test recovery readiness after runtime-level RCE, and examine whether AI-assisted vulnerability discovery is outpacing the remediation capacity of foundational open-source projects.

Method and Evidence Note

This is a rapid incident-analysis report based on publicly available primary and secondary sources reviewed through 1 November 2026. Fast-moving incident reporting can change as vendors validate or retract telemetry. Where sources conflict, this report prioritizes PHP's official advisory and PHP Foundation statements for project facts, and distinguishes exploit attempts from confirmed successful compromise. No exploit was executed for this report.

References

[1] PHP Project. Security advisory: Critical remote code execution vulnerabilities, update now; PHP 8.5.12/8.4.27/8.3.36/8.2.35 releases, 21 Oct 2026. https://www.php.net/archive/2026.php

[2] The PHP Foundation. Today's PHP Security Release, and How We're Strengthening PHP Security, 21 Oct 2026. https://thephp.foundation/blog/2026/10/21/todays-php-security-release-and-how-were-strengthening-php-security/

[3] Wiz Research. PHP2Shell: AI uncovers critical RCE vulnerabilities affecting the PHP ecosystem, Oct 2026. https://www.wiz.io/blog/php2shell-php-rce

[4] Palo Alto Networks Unit 42. Threat Brief: Critical Vulnerabilities in PHP: CVE-2026-109618, CVE-2026-109619, updated 29 Oct 2026. https://unit42.paloaltonetworks.com/cve-2026-109618-cve-2026-109619/

[5] Sophos. Advisory: PHP2Shell Vulnerabilities (CVE-2026-109618, CVE-2026-109619), Oct 2026. https://www.sophos.com/en-us/security-advisories/sophos-sa-20261021-php2shell

[6] The PHP Foundation. Announcing the Ecosystem Security Team at The PHP Foundation, 18 May 2026. https://thephp.foundation/blog/2026/05/18/announcing-ecosystem-security-team/

[7] The PHP Foundation. Welcoming Alexandre Daubois to The PHP Foundation, 14 Jul 2026. https://thephp.foundation/blog/2026/07/14/welcoming-alexandre-daubois/

[8] The PHP Foundation. Welcoming Daniel Scherzer to the Ecosystem Security Team, 9 Sep 2026. https://thephp.foundation/blog/2026/09/09/welcome-daniel-scherzer/

[9] The PHP Foundation. Welcoming Elizabeth Barron as the New Executive Director, 27 Feb 2026. https://thephp.foundation/blog/2026/02/27/welcoming-elizabeth-barron-new-executive-director/

[10] The PHP Foundation. Roman Pronskiy Leaves The PHP Foundation Board and Brent Roose Joins, 1 Sep 2026. https://thephp.foundation/blog/2026/09/01/welcome-brent-roose/

[11] Check Point Research. PHP2Shell Update: One in ten organizations impacted as Europe bears the brunt, 22 Oct 2026. https://blog.checkpoint.com/ai-security/php2shell-one-in-ten-organizations-impacted-as-europe-bears-the-brunt/

[12] Fortbridge. PHP2Shell CMS Exploits: WordPress, Joomla and Drupal, 30 Oct 2026. https://fortbridge.co.uk/research/php2shell-wordpress-joomla-drupal/

Authorship and Rights Statement

Author: Md Riyadh Hasan, Independent Researcher, Oxford, United Kingdom. Copyright © 2026 Md Riyadh Hasan. This report's original prose, synthesis, incident analysis and the Hasan PHP Dependency Resilience Framework (PDRF) are presented as the author's analytical work. Third-party names, trademarks, CVEs, vulnerability discoveries, technical findings and quoted or paraphrased source material remain attributable to their respective sources and owners. Copyright notices do not establish ownership of the underlying PHP2Shell vulnerabilities or third-party research.

15. Extended Evidence Analysis: People, Disclosure, Social Engineering and Honeypots

The timing of personnel changes around a security incident can look suspicious when viewed without context. For that reason, this report examined public PHP Foundation announcements, the Foundation's current structure, and publicly indexed professional-network material. The result does not support an accusation against any named person. Instead, the documented changes show an organization expanding security capacity during a period in which AI-assisted vulnerability reports were increasing sharply.



Figure 4. Documented PHP Foundation/security-team changes during 2026



Figure 4. Documented PHP Foundation and security-team changes during 2026. Source synthesis: PHP Foundation announcements.

15.1 Personnel changes and what they do - and do not - show

Elizabeth Barron became Executive Director in February 2026 while founding Executive Director Roman Pronskiy transitioned toward his growing JetBrains role and initially remained on the board. In May, the Foundation created an Ecosystem Security Team and selected Volker Dusch to lead the funded security effort. Alexandre Daubois joined in July to help triage, handle and fix security reports. Daniel Scherzer joined the Ecosystem Security Team in September for hardening work. In the same month, Roman Pronskiy's board term ended and Brent Roose became the JetBrains representative. Public LinkedIn material also corroborates the Foundation's announcement of Alexandre's security-focused appointment and Daniel's longstanding PHP involvement.

These facts are relevant organizational context, but temporal proximity is not causation. None of the reviewed PHP Foundation statements, official PHP material, indexed LinkedIn material, AWS/Telekom honeypot reporting, Wiz research or Unit 42 reporting provides evidence that these individuals intentionally enabled the attack, leaked exploit details, or participated in a social-engineering operation. Naming them as suspects would therefore exceed the evidence.

15.2 Social-engineering and insider hypotheses

A human-factor hypothesis remains legitimate as a research question because Amazon observed an exploitation attempt on 17 October while the vulnerabilities were still under embargo, and the PHP Foundation has described premature circulation of vulnerability information before the planned disclosure. Several mechanisms could theoretically explain early attacker knowledge: independent rediscovery, monitoring of public code changes, accidental information exposure, researcher-to-researcher sharing, compromise of a communication channel, social engineering, or an intentional leak. Public evidence available for this report does not distinguish among those possibilities.

Figure 5. Hypothesis assessment matrix as of 1 November 2026

Hypothesis	Assessment	Evidence position
H1 Structural/process failure	Supported	PHP Foundation post-incident account
H2 Premature disclosure accelerated activity	Plausible / partly supported	Pre-disclosure exploit attempt + early public PoCs
H3 Human/social-engineering pathway	Open hypothesis	No public evidence identifies such a pathway
H4 Insider involvement	Unsupported	No reliable evidence implicates PHP personnel
H5 Honeypot observation can reconstruct attack tir	Supported as method	AWS MadPot + Telekom honeypot observations

Figure 5. Hypothesis assessment matrix. 'Unsupported' means no reliable public evidence was found; it does not prove impossibility.

15.3 Honeypot evidence

Honeypots are especially valuable because they provide time-stamped observations without requiring speculation about personnel. AWS reported a PHP2Shell exploitation attempt in its MadPot environment on 17 October, four days before public disclosure, from an address associated with a commercial anonymization network. AWS explicitly stated that it remained unclear how the actor obtained exploit information. Telekom Security then reviewed its own sensors and found probes before disclosure but no exploit-like payload in the eight hours immediately preceding the advisories; its first exploit-like serialized payload arrived at 14:18 UTC on 21 October, about 45 minutes after the advisories went live.

Figure 6. Telekom Security honeypot observations around public disclosure

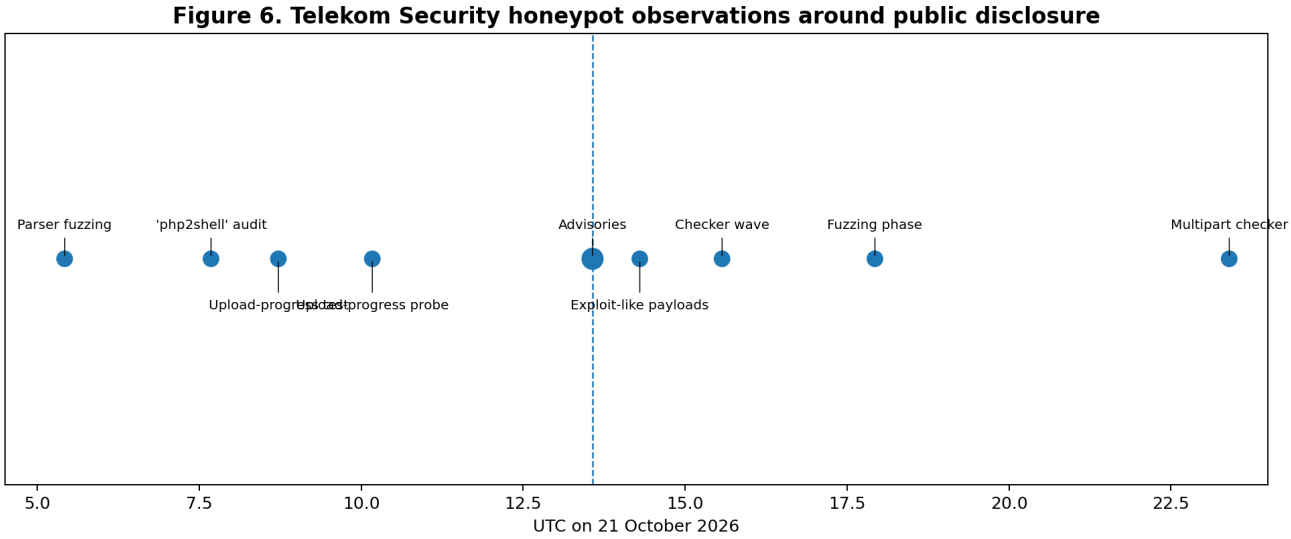


Figure 6. Telekom Security honeypot observations on 21 October 2026. The chart distinguishes probes from exploit-like traffic.

Taken together, the honeypot observations support two careful conclusions. First, at least one credible pre-disclosure exploitation attempt was observed by AWS. Second, widespread-looking scanning should not automatically be classified as successful exploitation: Telekom found many checkers and probes but only one source sending exploit-like payloads in the examined period. These findings strengthen the case for studying disclosure leakage while weakening simplistic claims of immediate mass compromise.

16. Visual Synthesis

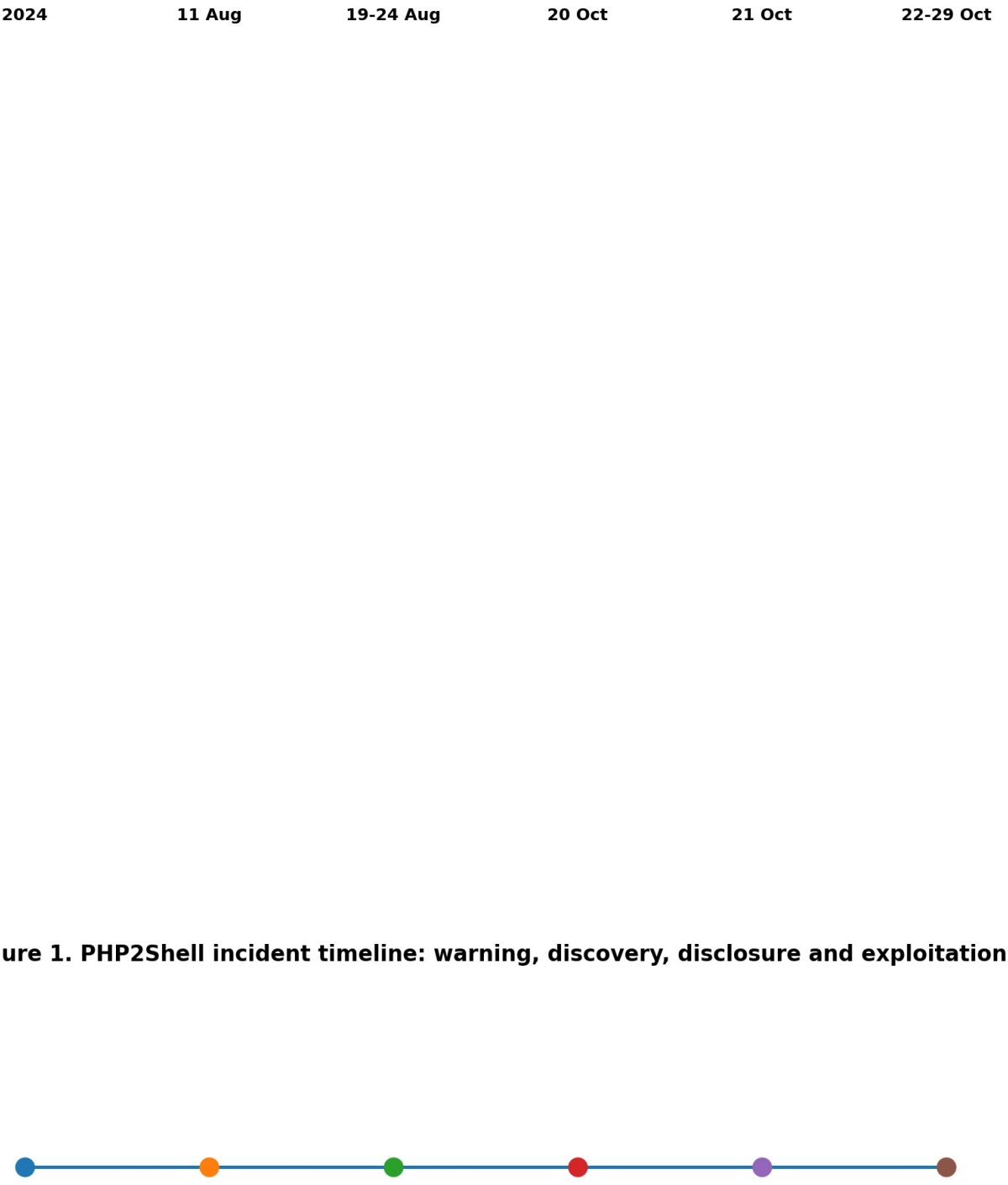


Figure 1. PHP2Shell incident timeline: warning, discovery, disclosure and exploitation

Figure 1. PHP2Shell incident timeline.

Figure 2. Defensive technical model of the PHP2Shell exploitation chain

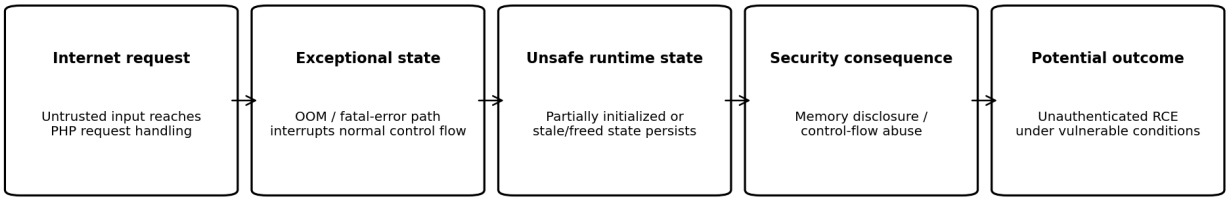


Figure 2. Defensive technical model of the vulnerability chain.

Figure 3. Evidence ladder used to separate findings from hypotheses

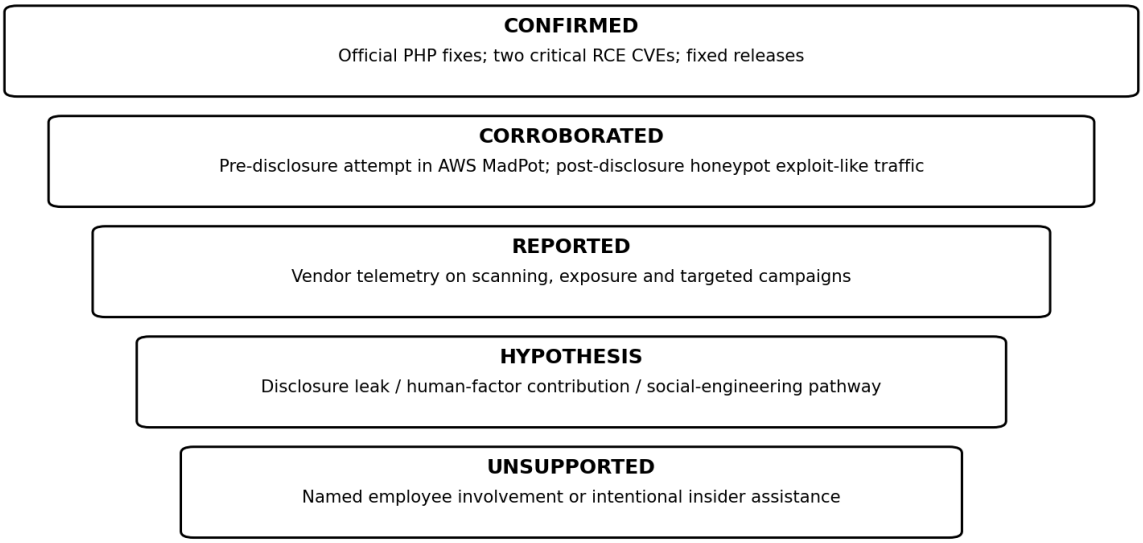


Figure 3. Evidence ladder used throughout this report.

Figure 7. PHP dependency concentration across internet-business services

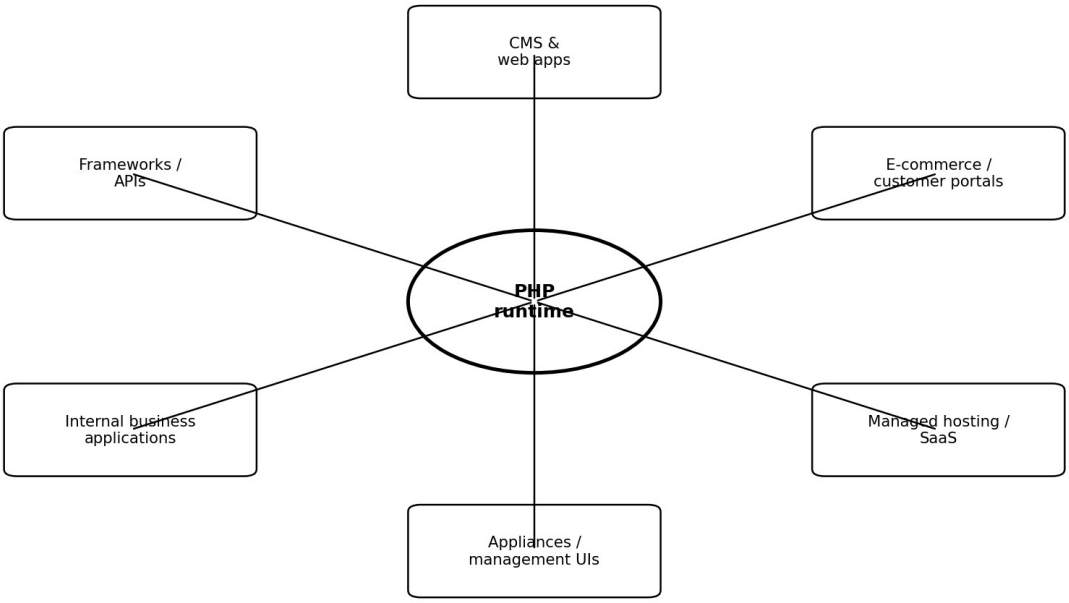


Figure 7. PHP dependency concentration across internet-business services.

Figure 8. How a runtime vulnerability can propagate into business loss

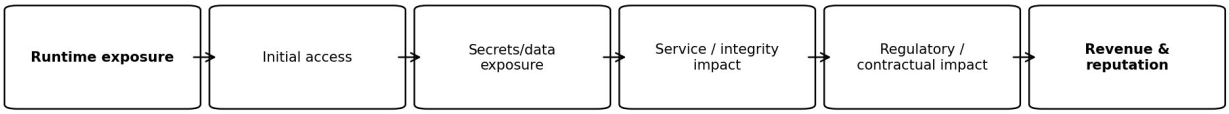


Figure 8. Business-impact propagation model.

Figure 9. Hasan PHP Dependency Resilience Framework (PDRF)

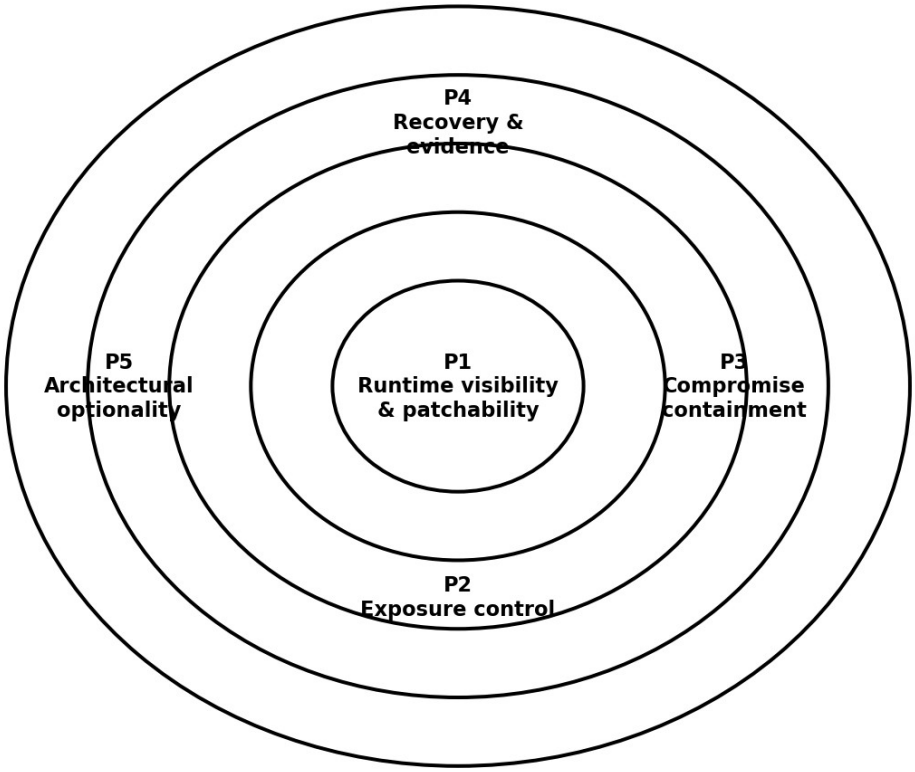


Figure 9. Hasan PHP Dependency Resilience Framework (PDRF).

Figure 10. Defensive response decision tree for PHP-dependent organizations

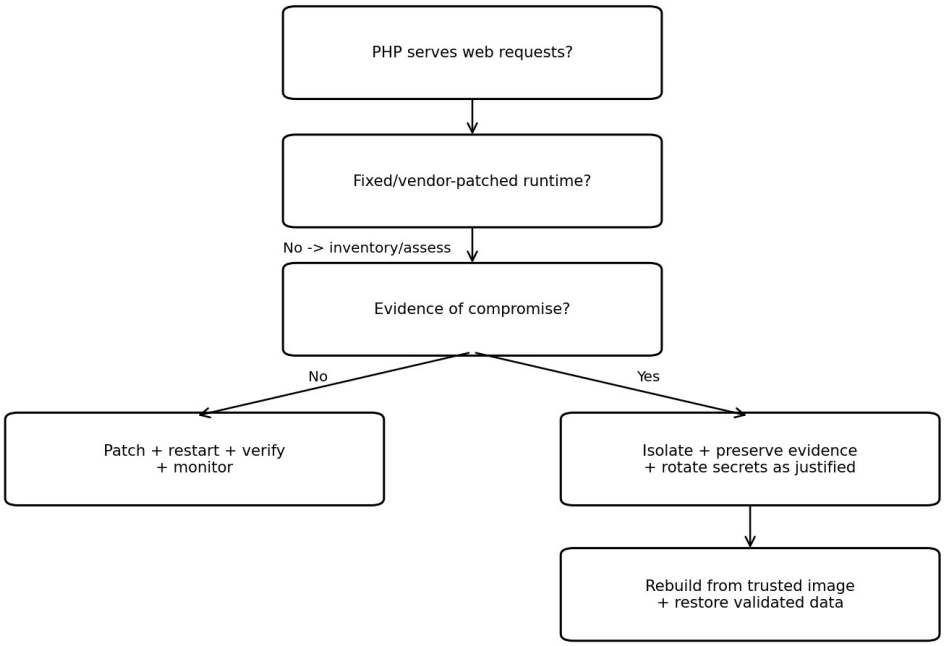


Figure 10. Defensive response decision tree.

17. Revised Hypotheses for Future Research

H1 - Structural/process failure: PHP2Shell severity was amplified by the interaction of a difficult runtime defect, delayed remediation, compatibility constraints, limited specialist capacity and vulnerability-management process. Current evidence supports this as the strongest explanatory hypothesis.

H2 - Premature disclosure accelerated hostile activity: Information escaping before coordinated disclosure increased the opportunity for scanning or exploitation. Pre-disclosure telemetry makes this plausible, although the causal path remains incompletely known.

H3 - Human/social-engineering contribution: A human communication or access pathway may have contributed to early attacker knowledge. This remains an open hypothesis, not a finding.

H4 - Insider involvement: A trusted participant intentionally supplied exploit information to attackers. No reliable public evidence reviewed for this report supports the claim, so it must remain unsupported unless new evidence emerges.

H5 - Honeypot reconstruction: Distributed honeypot telemetry can reconstruct the transition from generic probing to exploit-like activity and help distinguish disclosure-driven scanning from earlier knowledge. Existing AWS and Telekom observations support the usefulness of this method.

18. Proposed Safe Honeypot Study

A follow-up empirical study could deploy an isolated, non-production PHP honeypot using only systems owned or explicitly authorized by the researcher. The environment should contain no real

customer data, credentials or reusable secrets, and should be segmented so that a compromise cannot be used to attack third parties. The objective would be observation, not counterattack.

- Record first-seen timestamps for PHP2Shell-themed probes and exploit-like requests.
- Classify traffic as generic scan, vulnerability check, exploit-like request, confirmed execution in the sandbox, and post-exploitation behavior.
- Measure the interval between public disclosures/PoCs and changes in traffic volume or sophistication.
- Record source ASN and coarse country-level network origin where legally and ethically appropriate, without attempting to deanonymize individuals.
- Capture process creation, filesystem changes and outbound connection attempts inside the sandbox while blocking harmful egress.
- Publish only defensive indicators and aggregate statistics; avoid releasing weaponized exploit instructions.
- Pre-register classification rules so that unusual traffic is not automatically labelled PHP2Shell.

19. Final Assessment

The expanded evidence changes the emphasis of this report but not its core conclusion. PHP2Shell is first a critical runtime-security incident and second a case study in how technical debt, disclosure pressure, open-source governance and internet-scale dependency interact. The pre-disclosure AWS observation justifies asking how exploit knowledge escaped or was independently obtained. It does not justify accusing recently joined or departing PHP personnel. The most defensible research position is therefore to preserve the insider/social-engineering idea as a falsifiable hypothesis, actively look for contrary evidence, and use independent telemetry such as honeypots to test the timeline.

For organizations, the immediate lesson remains practical: patch the PHP runtime, restart and verify the serving processes, investigate exposed systems, isolate management interfaces, protect and rotate secrets when warranted, and maintain recoverable infrastructure. For researchers, the larger opportunity is to understand how security teams can manage AI-accelerated discovery and disclosure without allowing the remediation window to become an attacker advantage.

20. Additional Sources for the Extended Analysis

1. PHP Project. Security advisory / 2026 release archive. https://www.php.net/archive/2026.php/

2. The PHP Foundation. Today's PHP Security Release, and How We're Strengthening PHP Security. https://thephp.foundation/blog/2026/10/21/todays-php-security-release-and-how-were-strengthening-php-security/

3. The PHP Foundation. Welcoming Elizabeth Barron as the New Executive Director. https://thephp.foundation/blog/2026/02/27/welcoming-elizabeth-barron-new-executive-director/

4. The PHP Foundation. Announcing the Ecosystem Security Team. https://thephp.foundation/blog/2026/05/18/announcing-ecosystem-security-team/

5. The PHP Foundation. Welcoming Alexandre Daubois. https://thephp.foundation/blog/2026/07/14/welcoming-alexandre-daubois/

6. The PHP Foundation. Welcoming Daniel Scherzer to the Ecosystem Security Team. https://thephp.foundation/blog/2026/09/09/welcome-daniel-scherzer/

7. The PHP Foundation. Roman Pronskiy Leaves the Board and Brent Roose Joins. https://thephp.foundation/blog/2026/09/01/welcome-brent-roose/

8. AWS Security Blog. Amazon Threat Intelligence identifies PHP2Shell exploitation prior to public disclosure in MadPot. https://aws.amazon.com/blogs/security/amazon-threat-intelligence-identifies-php2shell-exploitation-prior-to-public-disclosure-in-madpot/

9. Telekom Security. Honeypot Observations on PHP Vulnerabilities CVE-2026-109618 and CVE-2026-109619. https://github.security.telekom.com/2026/10/php2shell-honeypot-observations.html

10. Wiz Research. PHP2Shell: AI uncovers critical RCE vulnerabilities. https://www.wiz.io/blog/php2shell-php-rce

11. Wiz Research. PHP2Shell: Exploitation Attempts and Remediation Trends. https://www.wiz.io/blog/php2shell-exploitation-so-far

12. Unit 42. Threat Brief: Critical Vulnerabilities in PHP. https://unit42.paloaltonetworks.com/cve-2026-109618-cve-2026-109619/

13. LinkedIn / PHP Foundation. Public announcement of Alexandre Daubois security appointment. https://www.linkedin.com/posts/phpfoundation_php-security-phpc-activity-7482810283477024769-m1E5